Day 15 Report: Agent Harness

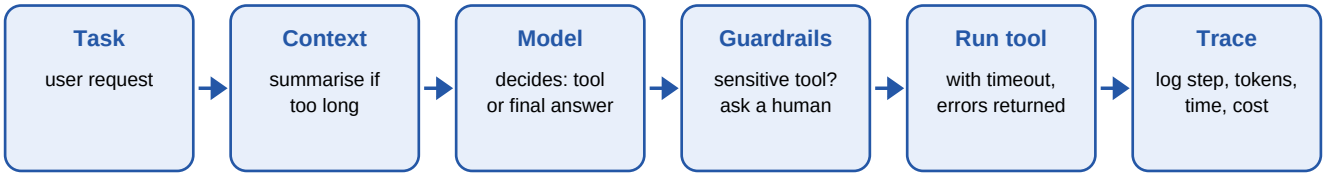
Turning the Day 14 agent into a reusable runtime: tools, budgets, memory, guardrails, tracing

1. Goal

The LLM is only the brain. A real agent also needs a **runtime around it** that decides which tools it can use, how long it can run, what happens when something breaks, what it remembers, which actions need a human's permission, and a record of everything it did. This runtime is the **agent harness** (Claude Code is a large example of one).

2. How it works

Harness loop (repeats until answer or budget)



Task part	What I built
1. Harness core	Tool registry (add a tool with one decorator line), configurable system prompt, step budget (8), token budget (40,000), per-tool timeouts, errors returned to the model
2. Context and memory	Old history is summarised by the LLM when it passes a limit (fallback: cut). Long-term memory: a JSON file with save_memory / recall_memory tools
3. Guardrails	Tools marked safe or sensitive; write_file and send_email (simulated) need human approval; run_python runs in a sandbox (separate process, 5 s CPU, 256 MB, no secrets, empty folder)
4. Tracing and comparison	Every run saved to traces.jsonl (steps, status, tokens, time, cost); the same agent rebuilt with LangGraph and compared

3. Tools

Tool	Type	Timeout
calculator, get_today, save_memory, recall_memory	Safe	15 s
web_search (DuckDuckGo, Wikipedia backup)	Safe	25 s
run_python	Safe, runs in sandbox	10 s
write_file, send_email (simulated, writes to outbox.txt)	Sensitive: needs approval	15 s
slow_tool (sleeps 10 s, only to test timeouts)	Safe	3 s

4. Results: one test per feature

Backend: Colab built-in Gemini (no API key), so tokens are estimated (characters / 4) and cost is $0.

Test	What happened	Feature shown
Eiffel Tower year, square root	web_search -> 1889, calculator -> 43.46	Normal multi-step task
Call slow_tool, else 2^20	slow_tool timed out after 3 s; agent switched to calculator -> 1,048,576	Timeout + error recovery
20th Fibonacci with Python	Wrote code, sandbox printed 6765	Sandboxed code
Remember two facts (session 1)	save_memory stored the deadline and favourite language	Long-term memory
"What is my deadline?" (new session)	recall_memory -> "Friday 14 November"	Memory across sessions
Send an email	Approval policy = deny; agent stopped and explained it could not send	Guardrail (deny)

Test	What happened	Feature shown
Write notes.txt	Agent searched, then asked; I typed y; file written inside agent_files/	Human-in-the-loop (approve)
5-step calculation, 150-token context limit	History summarised twice; all 5 results still correct (391, 736, 2208, 1208, 172.57)	Context summarising

Metric	Result
Runs that reached a final answer	8 of 8
Runs stopped by step or token budget	0
Tool timeouts handled without a crash	1 of 1
Sensitive actions denied / approved	1 / 1
Context summaries triggered	2 (in the long calculation)
Tokens per run (estimated)	about 1,800 to 6,200
Time per run	10 to 67 s (183 s for the run that waited for my approval)

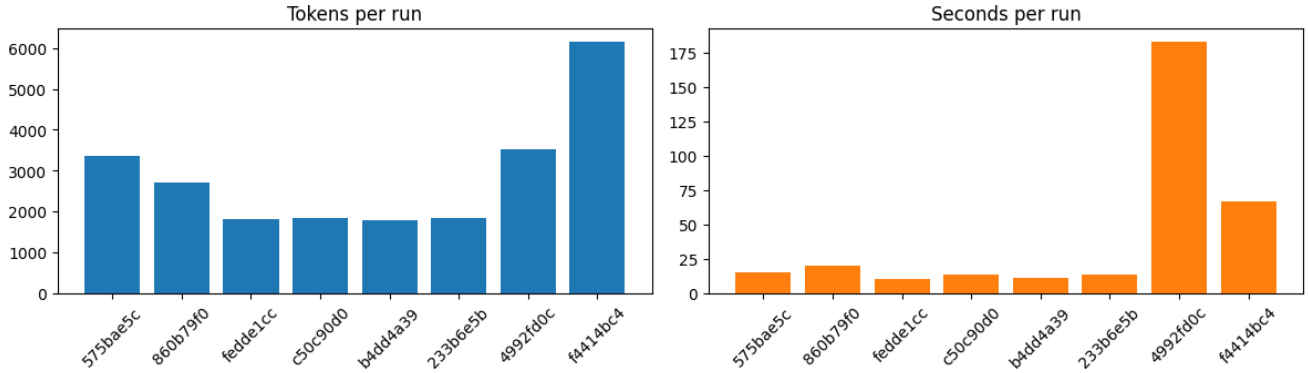


Figure 1. Tokens and seconds per run (from traces.jsonl)

5. My harness vs LangGraph

The same agent (same model, prompt and tools) was rebuilt as a LangGraph graph: a *model* node, a *tools* node and an edge back. It answered both test tasks correctly (43.46 and 6765).

	My harness	LangGraph
Lines of code (agent loop)	96	26
Step limit	max_steps, stops gracefully with a reason	recursion_limit, raises an error
Token budget, tool timeouts	built in	not built in
Errors back to the model	yes, with a status	only what I code in the node
Human approval	per tool (sensitive flag)	possible with interrupts, more setup
Context summarising	built in	not built in
Tracing	every run and step in traces.jsonl	needs LangSmith or custom code
Structure	plain Python loop	clear graph, easy to add branches

Verdict: LangGraph needs much less code and its graph is easy to read and extend. My own harness takes more code but gives full control and better debugging out of the box (budgets, timeouts, approvals, traces).

6. Key findings

- **All 8 runs finished with a correct final answer**; none hit the step or token budget.
- **Returning errors as text works:** after the timeout the agent read the error and changed plan.
- **Memory really survives sessions:** a brand-new harness answered from the JSON memory file.

- **Guardrails work both ways:** a denied email was not sent and the agent explained why; an approved file was written only inside agent_files/ (a "../" path cannot escape).
- **Summarising keeps prompts small** without losing the numbers needed for the answer.
- Traces make every step, token and second visible, which is what makes an agent debuggable.

7. Limitations and next steps

- Token counts are estimates (no API key); add a key to get exact tokens and real cost.
- Memory search uses word overlap; a vector store (like Day 13) would find facts by meaning.
- The sandbox does not block network access; production would use Docker or a VM.
- Day 16: add an MCP client so this harness can use tools from any MCP server.